

Room Occupancy Estimation: Student Guide

Prepared by Engr. Jamie Eduardo Rosal, MSCpE

A repeatable laboratory workflow aligned with the supplied AI proposal. Revised edition: time-aware evaluation, controlled sensor ablation and cautious interpretation.

How to use this guide

Work through steps 1-12 in order. Read the action and checkpoint, run the matching notebook cell, then inspect its output. The guide provides instructions; the notebook contains executable code. The tables below are generated from notebook outputs, not typed manually.

1. Understand the objectives
2. Prepare the environment
3. Load and validate the dataset
4. Explore the training data
5. Define the controlled experiments
6. Create and audit the splits
7. Train and evaluate all configurations
8. Inspect errors and ablation changes
9. Analyze feature and group importance
10. Assess robustness across days
11. Write evidence-based conclusions
12. Reproduce and submit

1. Understand the objectives

Goal: classify whether a room contains 0, 1, 2, or 3 people using environmental measurements. This is multiclass classification, not continuous regression.

Objective 1: develop the full-feature Random Forest. Objective 2: remove each of five sensor groups and retrain. Objective 3: compare ablation effects with individual and grouped feature importance.

A Random Forest combines decision trees. Ablation means removing one group, retraining, and comparing on exactly the same test rows. The proposal title is Random Forest-Based Sensor Ablation Analysis for Environmental Feature Contribution in Multiclass Room Occupancy Estimation.

Checkpoint: explain why the occupancy label must never be an input. This dataset cannot establish performance on occupancy counts outside 0-3.

2. Prepare the environment

Materials: laptop/desktop, Python 3.12, JupyterLab, and the supplied `Occupancy_Estimation.csv`. A practical recommendation is 8 GB RAM, not a measured minimum. Internet is needed for installation only.

Open a terminal in this project folder. macOS/Linux: run `python3.12 -m venv .venv` and then source `.venv/bin/activate`. Windows Command Prompt: run `py -3.12 -m venv .venv` and then `.venv\Scripts\activate.bat`.

Run `python -m pip install -r requirements.txt`. Register the environment with `python -m ipykernel install --user --name occupancy-guide --display-name "Occupancy Guide"`. Launch `python -m jupyterlab`, open `Room_Occupancy_Estimation.ipynb`, and select the Occupancy Guide kernel.

Use the same Python for installation and execution. This avoids the system-Python versus Anaconda mismatch encountered in the initial build. The local CSV makes `ucimlrepo` unnecessary.

Action: run the cell. Expected: Python executable and package versions. Checkpoint: resolve import errors before continuing; do not suppress warnings globally.

Commands after activating the environment

```
python -m pip install -r requirements.txt
```

```
python -m ipykernel install --user --name occupancy-guide --display-name "Occupancy Guide"
```

```
python -m jupyterlab
```

3. Load and validate the dataset

Action: read the CSV and define the sensor groups explicitly. There are 19 columns: Date, Time, 16 sensor features and the occupancy target. Sixteen features do not mean sixteen physical sensors.

Temperature: S1-S4 Temp. Light: S1-S4 Light. Sound: S1-S4 Sound. CO2: S5_CO2 and S5_CO2_Slope together. PIR: S6_PIR and S7_PIR. Consult UCI documentation for units/calibration rather than assuming them from column names.

Use Date/Time only for ordering and evaluation, not prediction. Preserve source-row identifiers. Missing targets, invalid timestamps, unexpected labels or duplicate timestamps stop execution. Remove exact duplicate rows before splitting and count them.

Missing sensor readings are allowed and later median-imputed using training data only. Infinite values become missing. Random Forest does not require scaling.

Expected: 10,129 rows, four labels and no missing values for the supplied CSV. Checkpoint: read the audit before accepting any result.

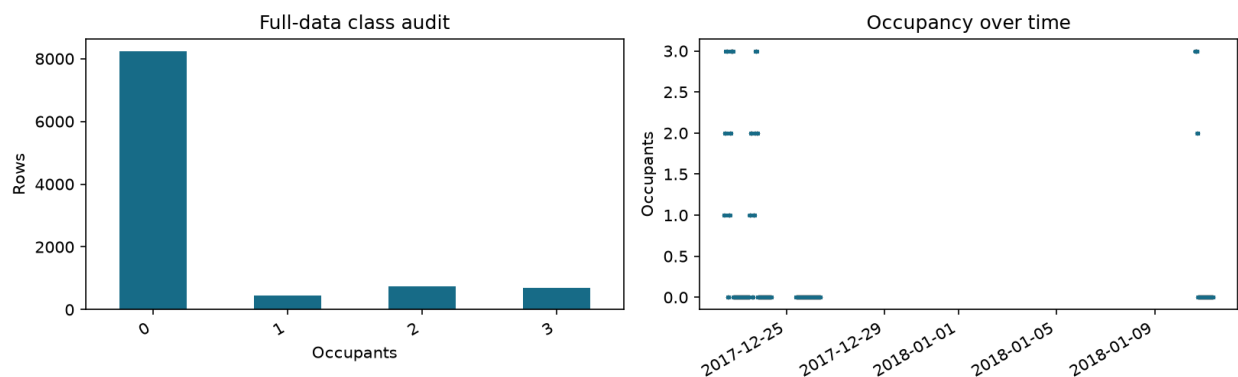
timestamp	0	1	2	3
2017-12-22	637	205	257	363
2017-12-23	1997	254	398	130
2017-12-24	1064	0	0	0
2017-12-25	1716	0	0	0
2017-12-26	1063	0	0	0
2018-01-10	703	0	93	201
2018-01-11	1048	0	0	0

4. Explore the training data

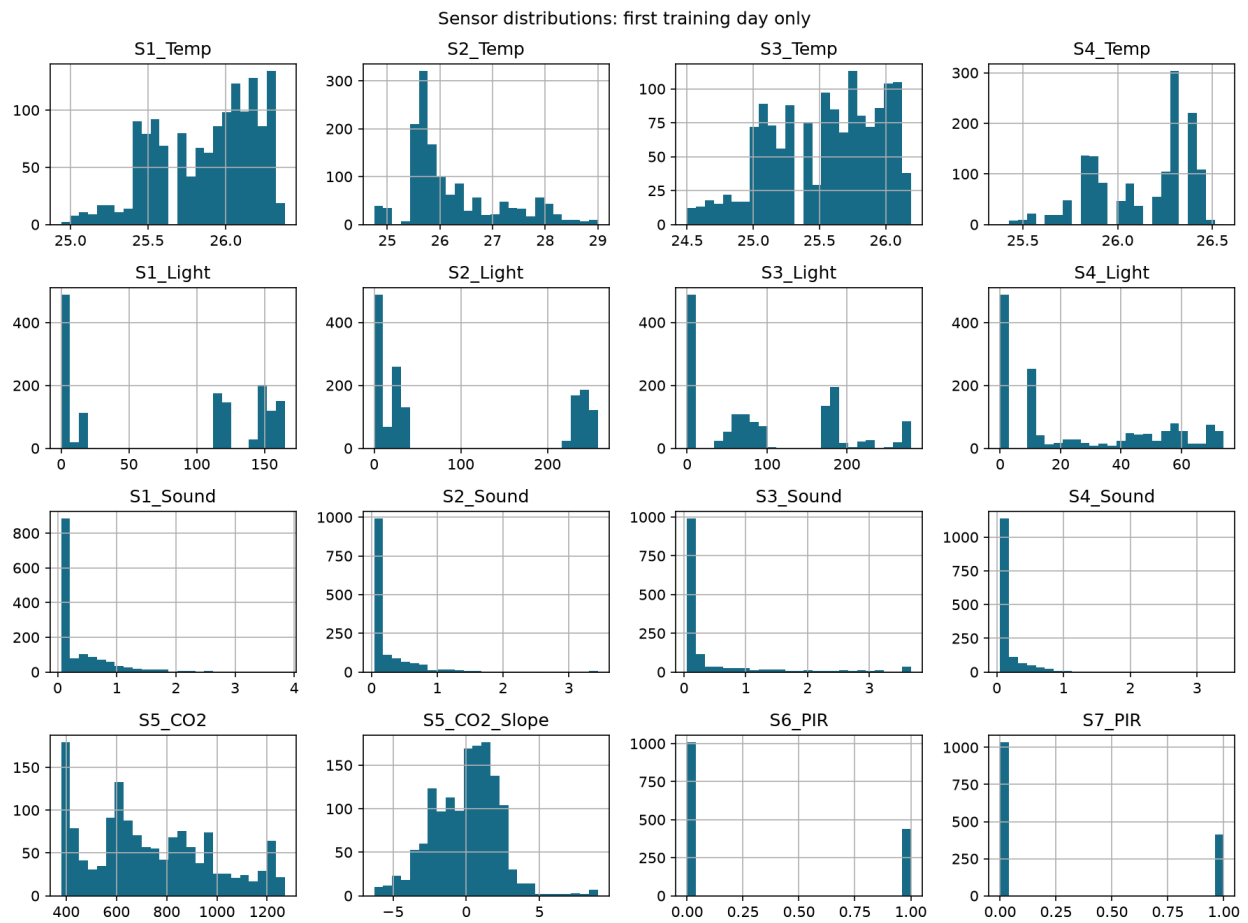
Action: inspect full-data class balance and timing as an audit. Use the first recorded day for detailed sensor exploration: it is training data in every temporal evaluation. Do not use later feature patterns to tune the model.

Plot sensor distributions and Pearson correlations. Count readings beyond 1.5 times the interquartile range (IQR), but do not delete them automatically. Genuine occupied periods may look unusual relative to empty periods.

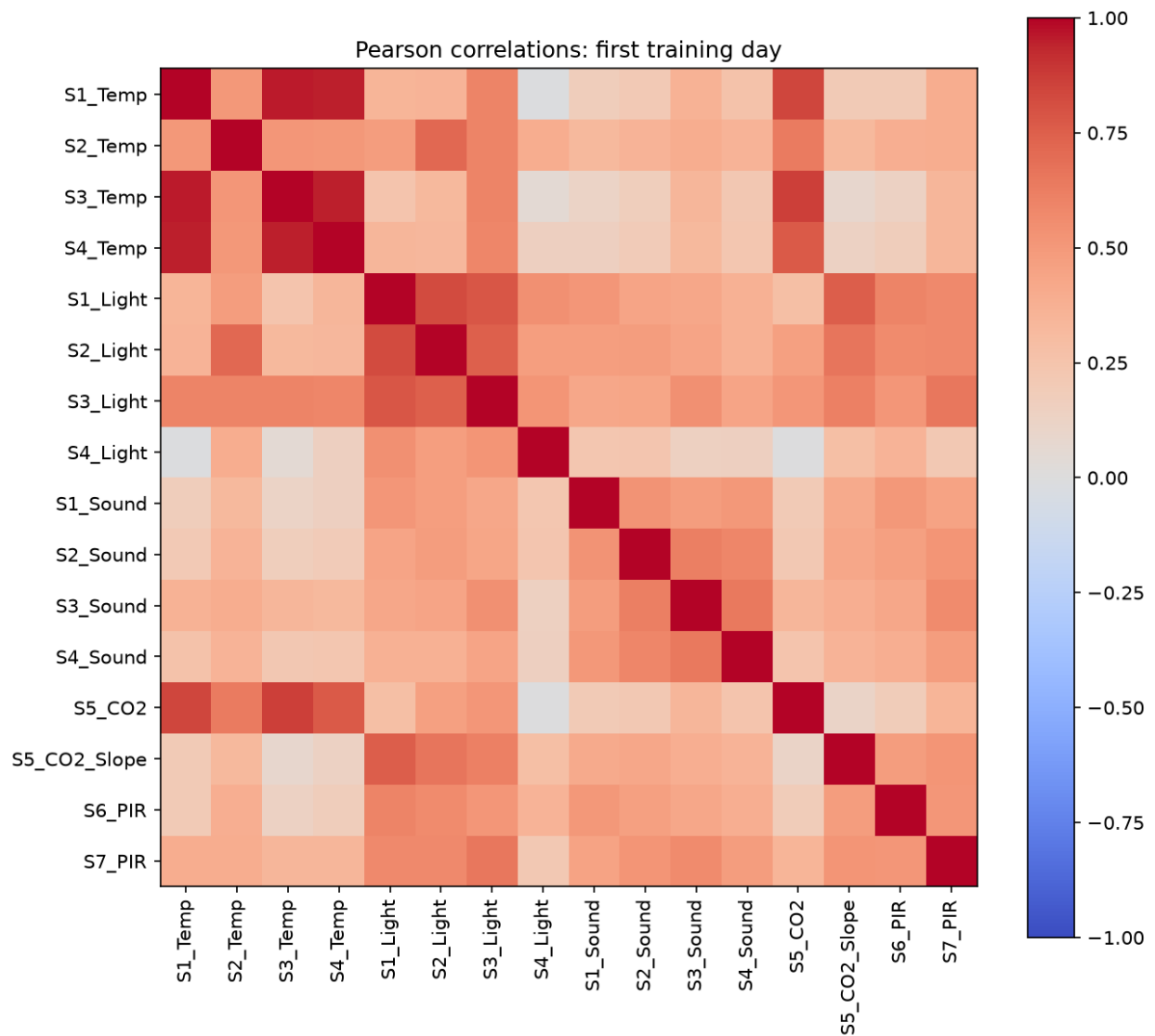
Expected: empty-room readings dominate (about 81%). Several dates contain only class 0, and January has no class 1. Checkpoint: explain why accuracy alone is insufficient and why test-class coverage matters.



Class and timing audit. Recording gaps are not interpolated.



First-day training distributions; unusual readings are retained.



Correlation is descriptive, not causal.

5. Define the controlled experiments

Action: compare All features with Without Temperature, Without Light, Without Sound, Without CO2, and Without PIR. A fresh pipeline is fitted for every experiment using identical split rows and model settings.

Use 100 trees, maximum depth 10, random seed 42, and max_features='sqrt', retaining the original settings. These are fixed instructional settings, not selected using test scores. Other key defaults are explicit below.

The sqrt rule is constant, but its effective candidate-feature count can change after removing columns. Ablation measures removal within this learning procedure, not a pure causal effect of a physical sensor.

Checkpoint: CO2 removal must include its slope. Date, Time, source_row, timestamp and the target must never enter X.

6. Create and audit the splits

Action: preserve the stratified random 80:20 split (seed 42) as a historical comparison, not evidence of future-session generalization. Neighboring readings can appear on both sides and are highly dependent.

Temporal holdout: train on December, test on January. This uses the natural multi-day recording gap. January has only classes 0, 2 and 3, so class-1 generalization cannot be assessed. No test scores are used to tune models.

Robustness: train on expanding past history and test each next recorded day, with a 10-minute embargo before that day's first test reading. This illustrative gap reduces immediate adjacency; it does not prove independence.

No fold is selected by performance. Separate occupied-day and empty-only-day summaries. Folds share training history and are not independent replications. The January holdout overlaps the final two daily tests, so these are alternative summaries, not independent confirmation.

Checkpoint: assert disjoint rows, temporal ordering and class coverage. Save exact source-row assignments. The cutoff and embargo are study design choices specific to this dataset, not optimized settings.

split	train_rows	test_rows	test_classes
random_80_20	8103	2026	0,1,2,3
temporal_holdout	8084	2045	0,2,3
daily_2017-12-23	1444	2779	0,1,2,3
daily_2017-12-24	4224	1064	0
daily_2017-12-25	5305	1716	0
daily_2017-12-26	7003	1063	0
daily_2018-01-10	8084	997	0,2,3
daily_2018-01-11	9062	1048	0

7. Train and evaluate all configurations

Action: run 48 small forests, six configurations across eight splits. Runtime depends on your machine. The median imputer is fitted inside each training pipeline; no scaling or oversampling is required.

Accuracy is the fraction correct. Precision asks how often a predicted class is correct; recall asks how often an actual class is found. F1 balances precision and recall. Weighted averages weight by class support and are dominated by class 0 here. Weighted recall equals accuracy in this single-label task.

Macro-F1 is the arithmetic mean of per-class F1 values. Use the classes present in each test split for the primary within-split ablation comparison. Also export fixed-four-class macro-F1, assigning zero to absent classes, for transparency. The report label macro avg fixed4 explicitly identifies that alternative average. Never compare macro scores across splits without considering class coverage. A zero-support class is untested, not a measured failure.

Export per-class reports, confusion matrices and exact predictions for every experiment. Checkpoint: each confusion matrix sums to the test-row count. Do not retrain just to print a report.

random_80_20

Configuration	Accuracy	Weighted F1	Macro F1 observed	Macro drop pp
All features	0.9970	0.9970	0.9903	+0.000
Without Temperature	0.9970	0.9970	0.9903	+0.000
Without Light	0.9916	0.9916	0.9709	+1.933
Without Sound	0.9980	0.9980	0.9938	-0.350
Without CO2	0.9975	0.9975	0.9920	-0.175
Without PIR	0.9975	0.9975	0.9920	-0.175

temporal_holdout

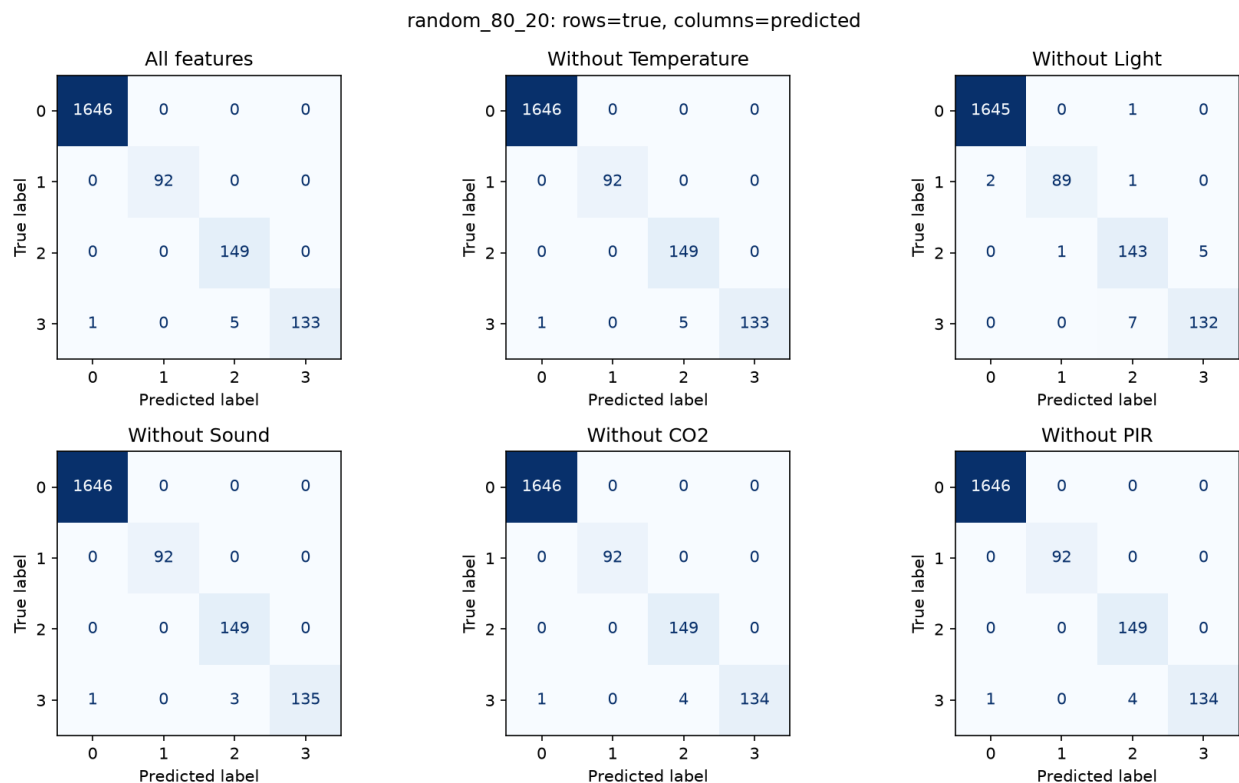
Configuration	Accuracy	Weighted F1	Macro F1 observed	Macro drop pp
All features	0.8802	0.8482	0.5288	+0.000
Without Temperature	0.8694	0.8335	0.4222	+10.656
Without Light	0.8812	0.8787	0.4649	+6.386
Without Sound	0.8826	0.8512	0.5478	-1.906
Without CO2	0.8822	0.8504	0.5441	-1.530
Without PIR	0.8822	0.8504	0.5441	-1.530

8. Inspect errors and ablation changes

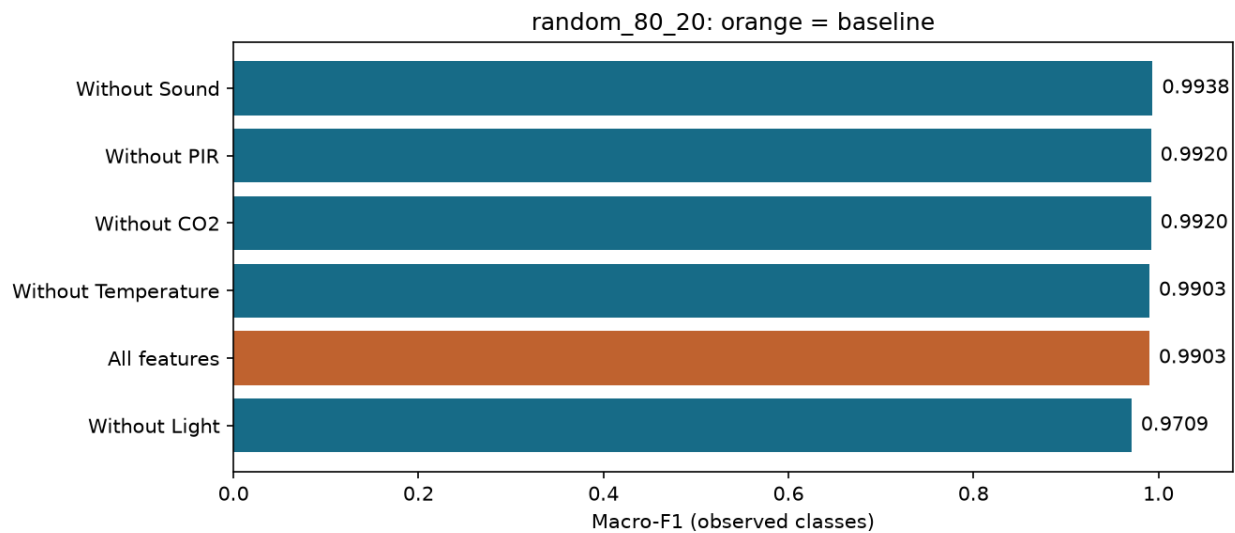
Action: inspect all six confusion matrices for the random comparison and temporal holdout. Rows are true labels, columns predictions. Off-diagonal entries are mistakes; an empty true-label row means there were no examples, not perfect recall.

Positive drop means removal worsens performance; negative means improvement. Drops use percentage points: 99.70% minus 99.16% is about 0.54 percentage points, not a relative decrease of 0.54%.

Checkpoint: examine occupied-class errors rather than just correct-empty-room counts. The baseline bar is colored by name, regardless of sorting. Full per-class reports for all configurations are saved in `classification_reports.json`.



random_80_20: all six configurations; zero-support rows are untested.

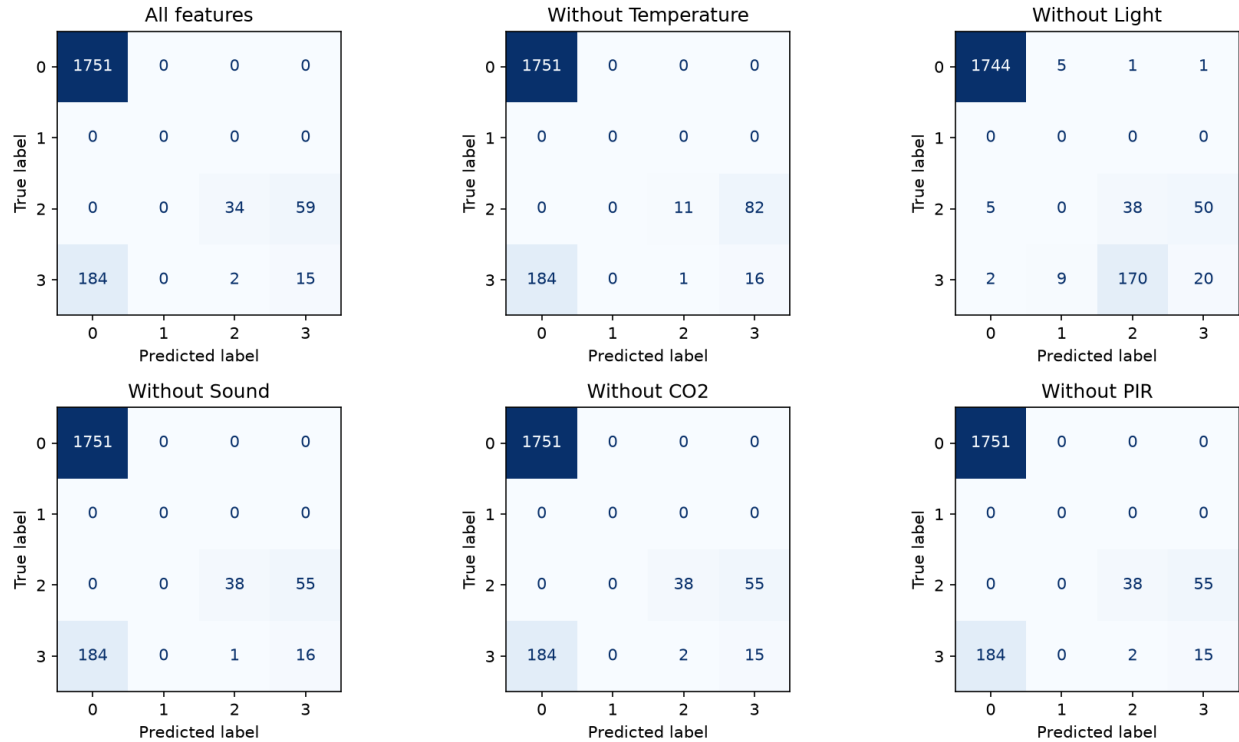


random_80_20: orange highlights the baseline.

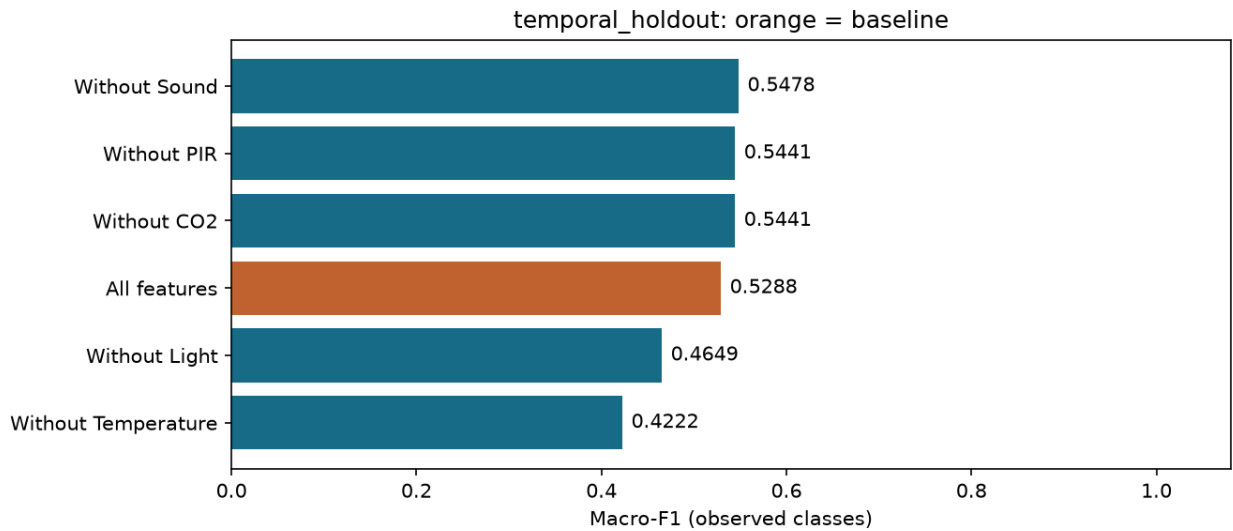
random_80_20 baseline per-class scores

Class	Precision	Recall	F1	Support
0	0.9994	1.0000	0.9997	1646
1	1.0000	1.0000	1.0000	92
2	0.9675	1.0000	0.9835	149
3	1.0000	0.9568	0.9779	139

temporal_holdout: rows=true, columns=predicted



temporal_holdout: all six configurations; zero-support rows are untested.



temporal_holdout: orange highlights the baseline.

temporal_holdout baseline per-class scores

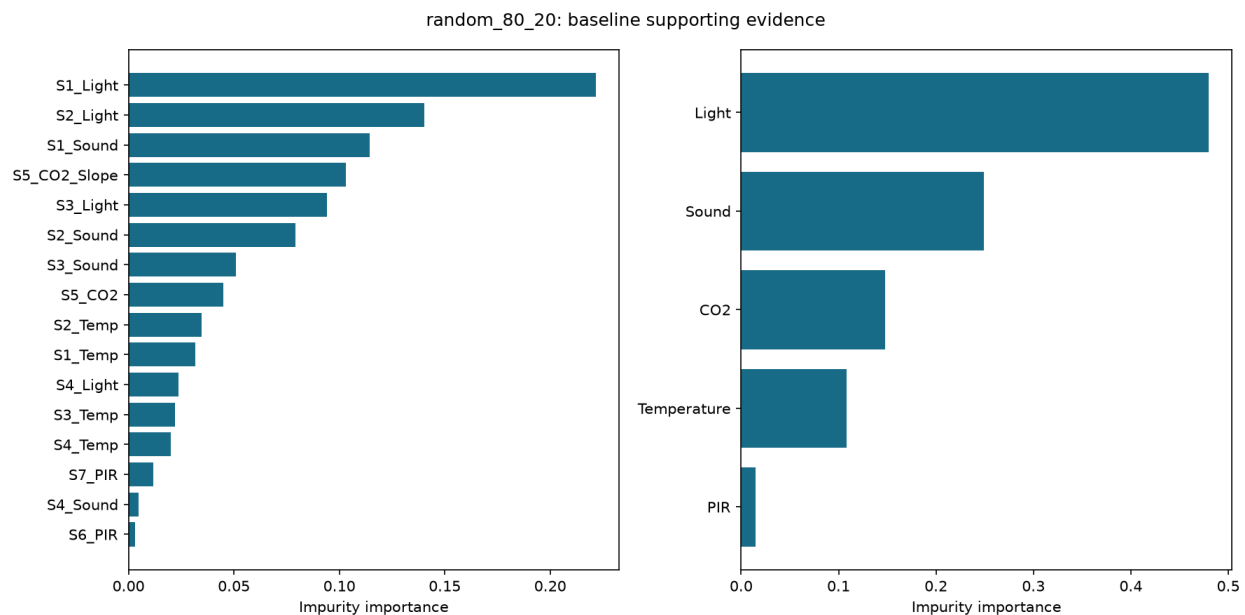
Class	Precision	Recall	F1	Support
0	0.9049	1.0000	0.9501	1751
1	0.0000	0.0000	0.0000	0
2	0.9444	0.3656	0.5271	93
3	0.2027	0.0746	0.1091	201

9. Analyze feature and group importance

Action: rank baseline impurity-based feature importance and sum within groups for both principal splits. These describe the trained forest, not causality or standalone sensor usefulness.

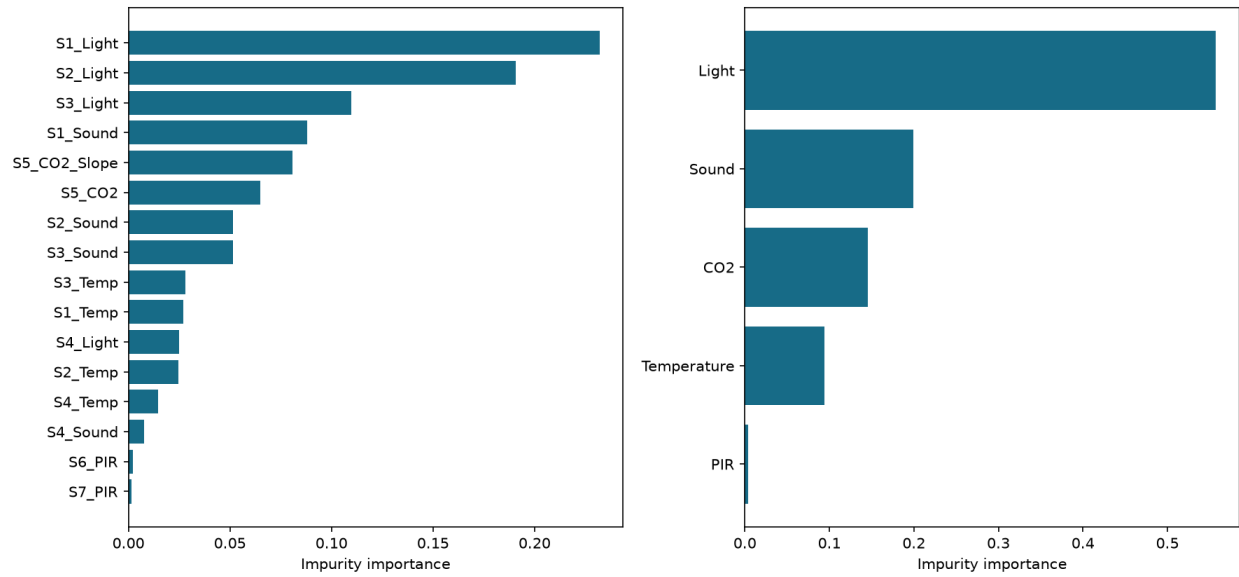
Impurity importance can favor features with many distinct values. Correlated inputs may substitute for one another. A feature can be important in the full model yet have a small ablation penalty because retraining lets other inputs replace it. Group sums also depend on group size.

Checkpoint: use ablation as main conditional-contribution evidence and importance as support. Do not declare Sound noisy merely because removing it improves a score.



random_80_20: individual-feature and group-level supporting evidence.

temporal_holdout: baseline supporting evidence



temporal_holdout: individual-feature and group-level supporting evidence.

10. Assess robustness across days

Action: summarize paired ablation drops within expanding-history daily folds. Report mean, sample standard deviation, minimum and maximum, separating occupied-present from empty-only days. Days are weighted equally, unlike pooled row-level scoring.

Only two daily test folds include occupied readings; four are empty-only. Test classes differ and training history overlaps. The spread is descriptive variability, not a confidence interval or statistical significance test. Repeating random seeds alone would not resolve temporal leakage.

Checkpoint: does the ranking change between random and temporal evaluations? Do not claim a universal most/least essential group when the evidence differs. Future work needs more independent occupied sessions and verification that the supplied CO2 slope is computed causally for real-time use.

empty-only daily folds

Configuration	Folds	Mean drop pp	SD pp	Range pp
All features	4	+0.000	0.000	+0.000 to +0.000
Without CO2	4	+0.000	0.000	+0.000 to +0.000
Without Light	4	+0.489	0.828	+0.000 to +1.719
Without PIR	4	+0.000	0.000	+0.000 to +0.000
Without Sound	4	+0.000	0.000	+0.000 to +0.000
Without Temperature	4	+0.000	0.000	+0.000 to +0.000

occupied-present daily folds

Configuration	Folds	Mean drop pp	SD pp	Range pp
All features	2	+0.000	0.000	+0.000 to +0.000
Without CO2	2	-1.569	0.055	-1.608 to -1.530
Without Light	2	+22.363	25.415	+4.391 to +40.334
Without PIR	2	-0.142	1.963	-1.530 to +1.246
Without Sound	2	-0.869	1.467	-1.906 to +0.169
Without Temperature	2	+4.379	8.876	-1.897 to +10.656

11. Write evidence-based conclusions

Action: report the exact split, test-class support, baseline scores and the largest/smallest observed ablation drops. Smallest does not prove dispensability. Zero or negative drops do not establish lack of information.

The initial random split classified 2,020 of 2,026 correctly. Removing Sound improved two predictions, not evidence of noise reduction. Compare the regenerated table instead of assuming time-separated evaluation preserves that ranking.

Limitations: one room, few recording days, temporal dependence, severe imbalance, absent class 1 in January, unverified real-time availability of CO2 slope and no external validation. This is offline classification, not a certified occupancy or safety system.

Checkpoint: address all three objectives while stating where evidence is insufficient. More occupied-session data is a recommendation, not an already completed experiment.

Computed findings

random_80_20: baseline accuracy=99.7038%, macro-F1 observed=0.9903; largest observed macro-F1 drop: Without Light (+1.933 pp); smallest: Without Sound (-0.350 pp). These are conditional effects, not universal sensor-necessity rankings.

temporal_holdout: baseline accuracy=88.0196%, macro-F1 observed=0.5288; largest observed macro-F1 drop: Without Temperature (+10.656 pp); smallest: Without Sound (-1.906 pp). These are conditional effects, not universal sensor-necessity rankings.

12. Reproduce and submit

Action: choose Restart Kernel and Run All Cells in Jupyter. Wait for completion and inspect outputs and warnings. The notebook overwrites its named outputs in results/. Never edit scores manually.

To rebuild the notebook and guides, run `python build_materials.py`. To regenerate guides after running the notebook, run `python build_materials.py --documents-only`. To execute twice and compare all numeric experiment outputs, run `python build_materials.py --verify`.

Keep requirements.txt, build_materials.py, notebook, CSV and results/ together. The DOCX opens in Word or imports into Google Docs; this does not create an online Google Doc. Older root-level reports and result CSVs are superseded by the current guide and results/.

Submission checklist: executed notebook; DOCX/PDF guide; checksum and versions; split audit/assignments; all six configurations per split; per-class reports/matrices; individual/group importance; robustness summaries; qualified conclusions.

Student questions: Why can random splitting inflate sensor-data performance? Why does weighted recall equal accuracy? How can a feature be important but have little ablation impact? Why is January class-1 recall untested? What further data supports deployment?

Troubleshooting: FileNotFoundError means working directory or CSV path is wrong. ModuleNotFoundError often means packages and kernel use different environments. Invalid labels/timestamps require investigation, not disabling assertions. PDF generation uses ReportLab and needs no LaTeX, browser or Pango.

Checkpoint: manifest and assertions must complete. Identical numeric results are expected in the same tested environment; different library versions can change results. The pinned file records direct dependencies, not every transitive package.

References: supplied Intro-to-AI-Proposal.docx; UCI Room Occupancy Estimation dataset, <https://archive.ics.uci.edu/dataset/864/room+occupancy+estimation> ; scikit-learn RandomForestClassifier, classification metrics and common pitfalls documentation, <https://scikit-learn.org/stable/> .

Dataset SHA-256

c090ee5c94b61762bfec9d2276786449 0b51030f98cca030f02822468db25d9c (join without the space)

Tested analysis versions: {"numpy": "2.4.6", "pandas": "3.0.1", "scikit-learn": "1.9.0", "matplotlib": "3.11.0"}